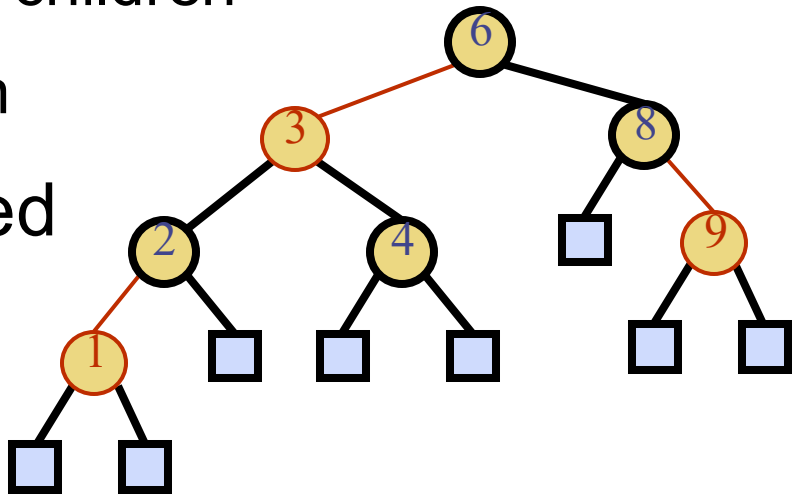
CS3021 Introduction to Data Structures and Intermediate Programming

Red-Black: A Self-Balancing BST

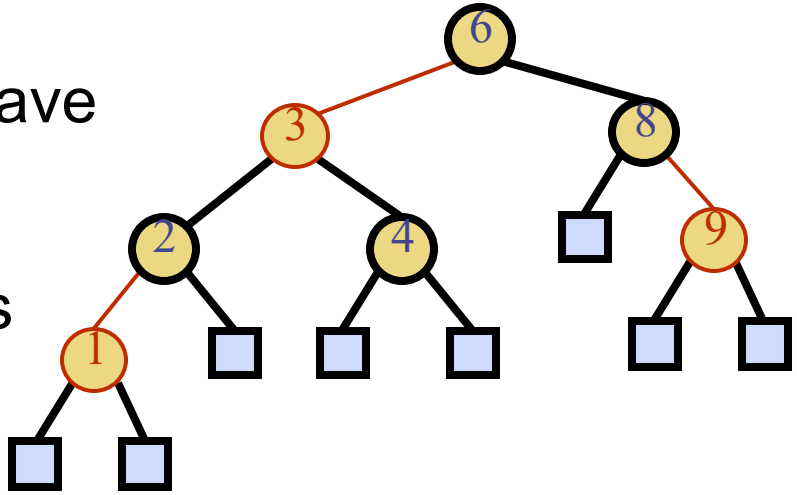
Red-Black Trees

- Red-Black Trees are self-balancing BSTs.
- As with any BST that allows efficient inorder traversal, search can be performed in *O(logN).*
- Two kinds of nodes:
  - Interior: hold data, have two children
  - Leaves: no data, no children
- Leaves can be implemented as an empty node, or a null pointer from parent.



Red-Black Tree Properties

- A red-black tree is a BST with the following properties:
  - Every node is colored either **red** or **black**.
  - Root Property: the root is black.
  - External Property: every leaf is black.
  - Internal Property: both children of a red node are black.
  - Depth Property: all leaves have the same *black height*, i.e., every path from a node to any of its descendant leaves contains the same number of black nodes.

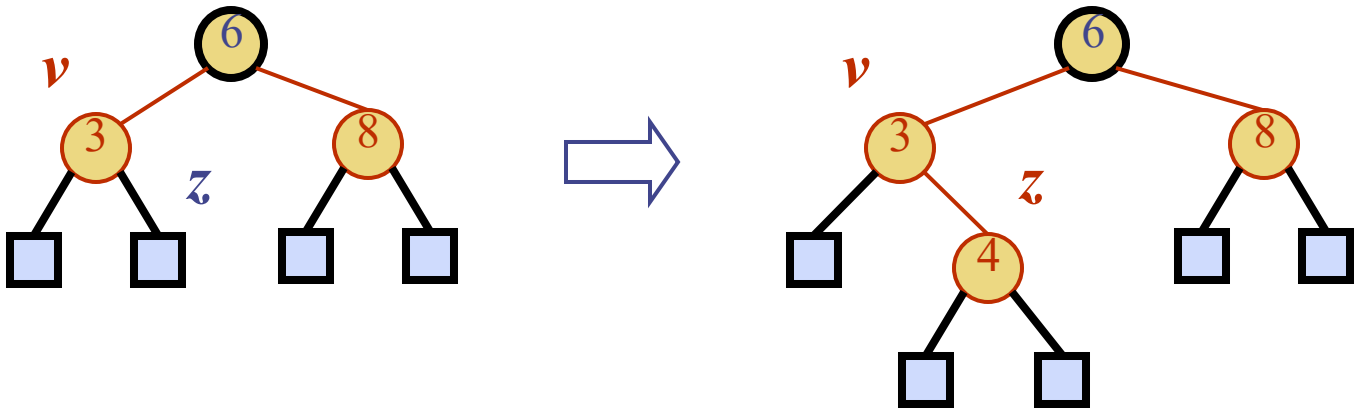


Red-Black Tree Insertion (1)

- To perform insert(***z***), we execute the insertion algorithm for a BST and color the newly inserted node ***z*** red, *unless* it is the root.
  - The *root*, *external*, and *depth* properties are preserved, since an additional red node has no impact on these properties.
  - If the parent ***v*** of ***z*** is black, we also preserve the *internal property,* and we are done;
  - Else parent ***v*** of ***z*** is is red, so we have a *double-red* (a violation of the *internal property*), which requires a reorganization of the tree.

Red-Black Tree Insertion Example

- Example where the insertion of 4 causes a *double-red* situation:



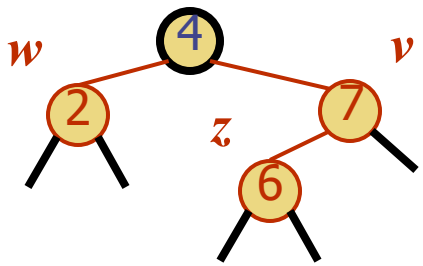
Red-Black Tree Insertion (2)

- Three possible cases for insert:
  - ***z***'s parent is red, and ***z***'s uncle ***w*** is also red.
  - ***z*** is in-facing child of red parent ***v***, and ***z***'s uncle ***w*** is a black node.
  - ***z*** is out-facing child of red parent ***v***, and ***z***'s uncle ***w*** is a black node.

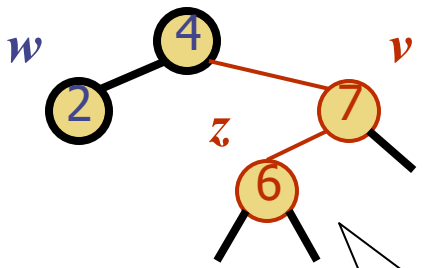
Remedying a Double-Red after Insert

- Consider a *double-red* with *child* ***z*** and *parent* ***v***, and let ***w*** be the *sibling* of ***v*** (the *uncle* of ***z***):

Case 1: child z's uncle w is red



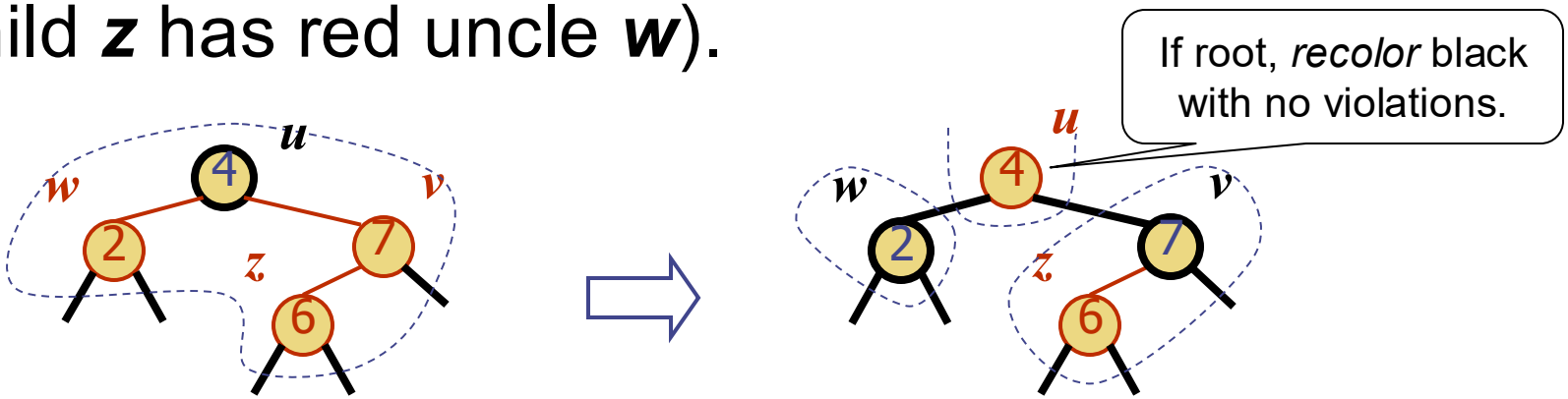
Case 2/3: child z's uncle w is black



If Case 2, simply rotate outward to create a Case 3.

Recoloring - Case 1

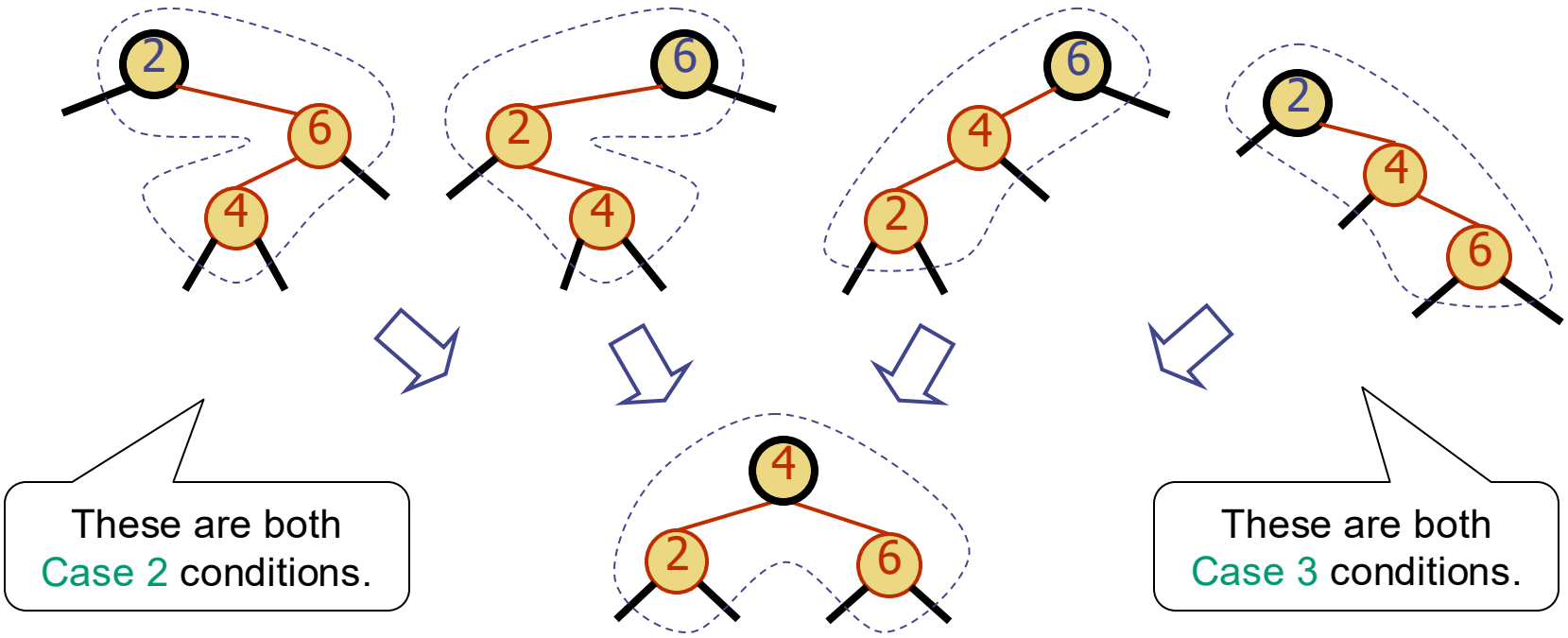
- A *recoloring* alone remedies a child-parent *double-red* when the parent has a red sibling (child ***z*** has red uncle ***w***).



- The parent ***v*** and its sibling ***w*** become black, and the grandparent ***u*** becomes red, unless it is the root (recall that root is always black).
- The *double-red* violation may propagate up to ***u***, if its parent is red; it becomes a new child ***z***.

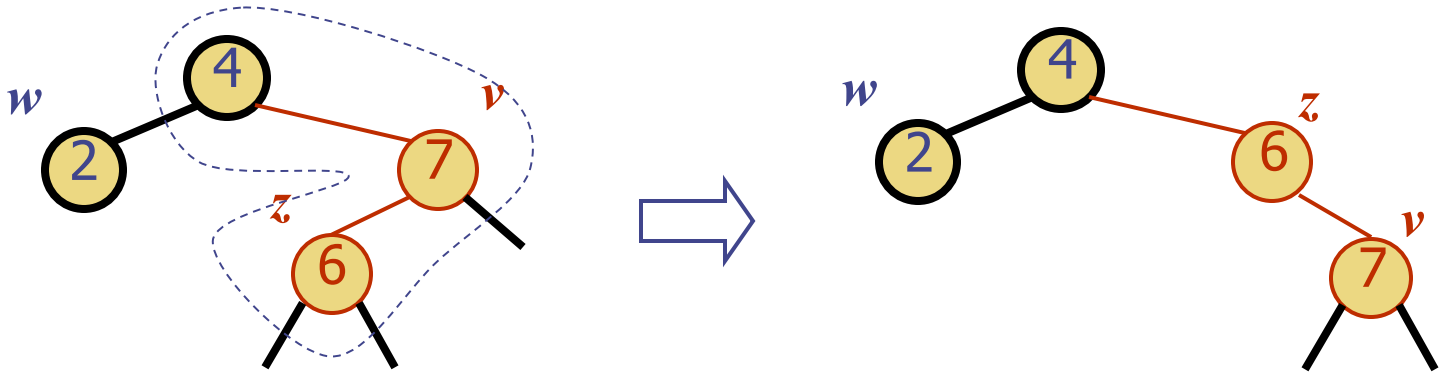
Restructuring - Case 2/3 (1)

- There are four restructuring configurations depending on whether the *double-red* nodes are left or right children:



Restructuring - Case 2/3 (2)

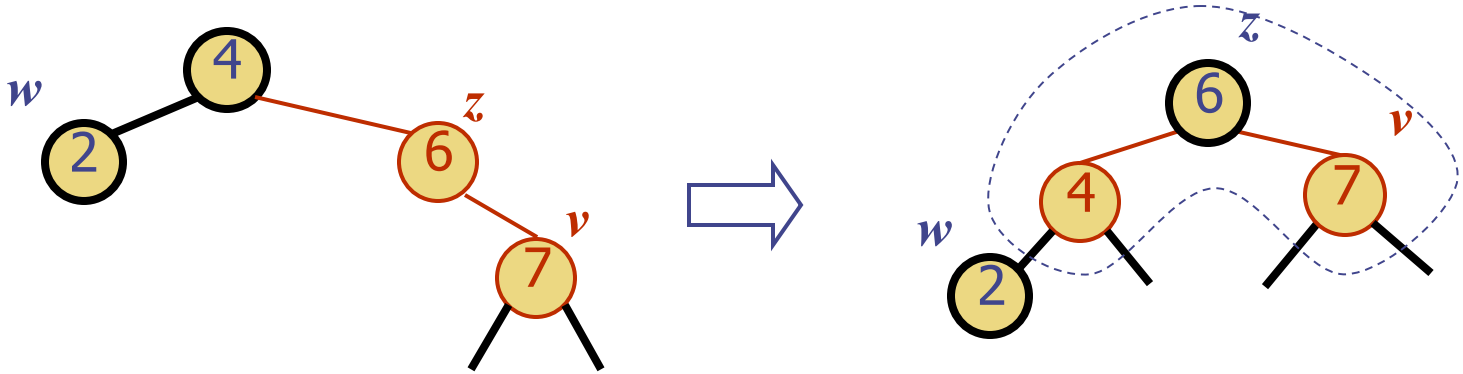
- A *restructuring* below remedies a child-parent *double-red* when the parent red node ***v*** has a black sibling (i.e., child ***z*** has black uncle ***w***).



- First, rotate ***z*** and ***v*** to create Case 3…

Restructuring - Case 2/3 (3)

- A *restructuring* below remedies a child-parent *double-red* when the parent red node ***v*** has a black sibling (i.e., child ***z*** has black uncle ***w***).



- … Then rotate ***z*** to sub-tree root, recolor ***z*** black and grandparent (old root) red.
- The *internal* property is restored, and the other properties are preserved.

RB Insertion Algorithm

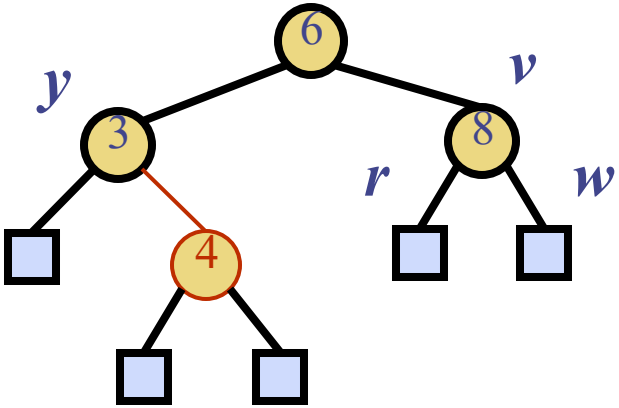
```
insert(k)
  use BSTsearch for key k to locate insert node z

  add new entry (key) at node z and color z red

  while doubleRed(z) {
     if isBlack(sibling(parent(z)))
        z = restructure(z)
        return
     else   //sibling(parent(z)) is red
        z = recolor(z)
  }
}
```

Red-Black Tree Deletion

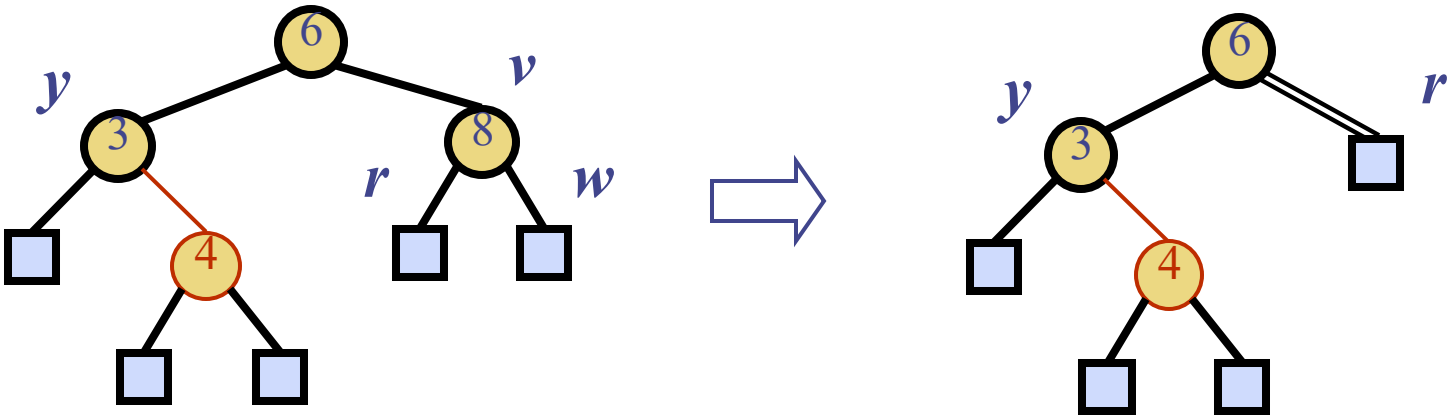
- To perform delete(***v***), we first execute the deletion algorithm for a BST.



- Let ***v*** be the internal node removed, ***w*** the external node removed, and ***r*** the sibling of ***w***.
  - If either ***v*** or ***r*** was red (but not both), we color ***r*** black, and are done;
  - Else (***v*** and ***r*** were both black), we color ***r*** "***double-black***", which requires a reorganization of the tree.

Red-Black Tree Deletion Example (1)

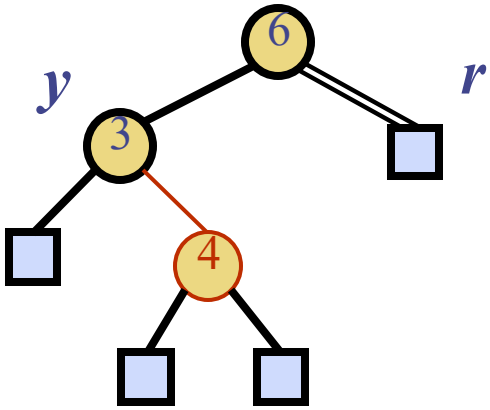
- In this example, the deletion of 8 creates a *double-black* condition.



- This is done to preserve the black connection (count) we lost when deleting node ***v***.

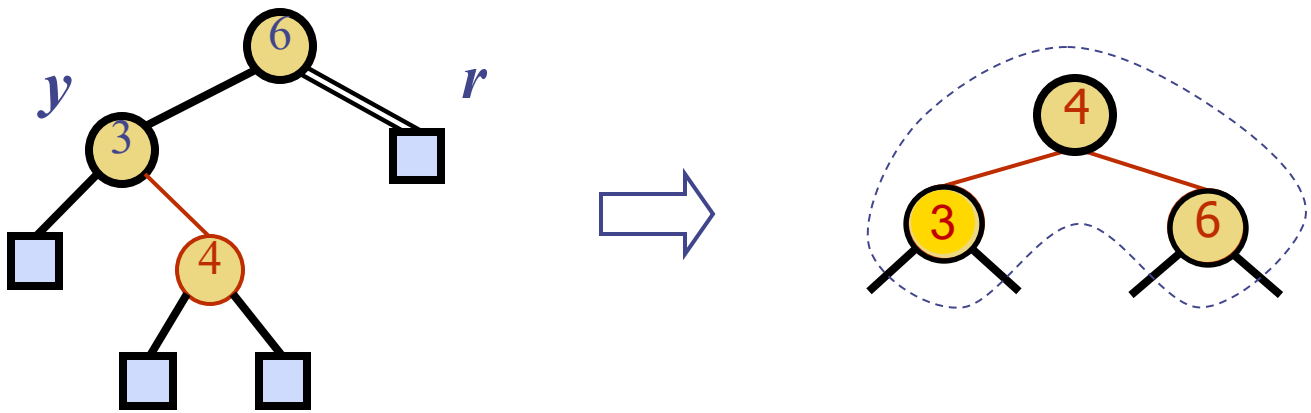
Remedying a double-Black

- The algorithm for remedying a *double-black* node ***r*** with sibling ***y*** considers three cases:
  - Case 1: ***y*** is black and has a red child:
    - Perform a restructuring, as with *insertion*, and we are done.
  - Case 2: ***y*** is black and its children are both black:
    - Perform a recoloring, which may propagate the *double-black* condition up the tree (as with *insertion*).
  - Case 3: ***y*** is red:
    - Perform an adjustment, equivalent to choosing a different representation of the tree, after which either Case 1 or Case 2 applies.



Red-Black Tree Deletion Example (2)

- In this example, we have a Case 1 condition on double-black node ***r*** (i.e., ***y*** is black and has a red child), so perform a restructuring, as with *insertion*:
  - Rotate ***y*** outward, then rotate its inside child (4 here) up to new root, and recolor as needed:



- And then we're done.

RB Deletion Algorithm

```
delete(k)
   use BSTsearch for key k to locate deletion node v
   BSTdelete (v)
   if v was red, color replacement node r black; done
   else, color sibling r double-black
   while doubleBlack(r) {
       if isBlack(sibling(r)) && isRed(child(sibling(r)))
           sibling(r) = restructure(sibling(r))
           return
       else if ((isBlack(sibling(r)) &&
                 !isRed(child(sibling(r)))))
           sibling(r) = recolor(sibling(r))
           return
       else   //sibling(r) is red
           sibling(r) = adjustment(sibling(r))
   }
}
```

Red-Black Tree Reorganization

Insertion	remedy double-red	
Red-black tree action	**(2,4) tree action**	**result**
restructuring	change of 4-node representation	double-red removed
recoloring	split	double-red removed or propagated up

Deletion	remedy double-black	
Red-black tree action	**(2,4) tree action**	**result**
restructuring	transfer	double-black removed
recoloring	fusion	double-black removed or propagated up
adjustment	change of 3-node representation	restructuring or recoloring follows

Red-Black Tree Performance

- Theorem: a red-black tree storing ***n*** entries has height $O(\log \boldsymbol{n})$.
- Thus ,the search algorithm for a red-black tree is the same as that for a BST.
- By the above, searching in a red-black tree takes $O(\log \boldsymbol{n})$ time.
- Deletion in a red-black tree also takes $O(\log \boldsymbol{n})$ time.

Questions?